NMJ 40203 Introduction to Data Analytics

Assignment 1

Group Assignment

Semester 2 2025/2026

NAME	MATRICS NUMBER
DINESH A/L MANIVANNAN	211021094
AKMAL HIDAYAT BIN ASRUL EFFENDI	211023322
IQRAM MASHRIQI BIN MARLIZAN	211023323

INTRODUCTION

In the current era of digital transformation, data has evolved into one of the most valuable assets across all domains of human activity. With the proliferation of internet-connected devices, sensors, social media platforms, and automated systems, the volume of data generated every second is staggering. However, raw data in itself holds limited value unless it is properly processed, interpreted, and utilized. This is where data analytics emerges as a critical discipline. Data analytics is the science of examining raw datasets in order to draw conclusions, discover patterns, and support informed decision-making.

At the heart of effective data analytics lies the process of data preparation, which includes tasks such as data cleaning, transformation, normalization, and feature engineering. High-quality analysis cannot be achieved without high-quality data, and this is why significant effort is devoted to handling missing values, correcting inconsistencies, and shaping raw data into a structured, analysable format. Furthermore, data analytics emphasizes the importance of clear communication, often achieved through data visualization techniques such as graphs, dashboards, and interactive reports. These tools not only make complex data more accessible, but also allow for real-time exploration of trends and patterns.

DATASET

The dataset titled “Household Energy Consumption – April 2025” represents a comprehensive and high-resolution collection of real-world energy usage records across multiple households over the span of a month. With a total of 90,000 records, this dataset captures essential information related to the daily patterns and behaviours of energy consumption in residential environments. Each row in the dataset corresponds to a specific observation, which could represent a household's daily energy footprint, and includes a variety of features that provide rich context for analytical exploration. Among the key variables are Energy_Consumption_kWh, which denotes the total electricity used in kilowatt-hours, and Peak_Hours_Usage_kWh, which highlights the volume of electricity consumed specifically during high-demand periods. These two features alone provide a meaningful distinction between base usage and demand-driven spikes, offering valuable insights into consumption trends that can inform energy optimization strategies.

In addition to quantitative energy metrics, the dataset also includes environmental and household demographic features, such as Avg_Temperature_C, which reflects the average external temperature for a given day, and Household_Size, which denotes the number of people living in each household. The presence of Has_AC, a binary categorical variable, further allows analysts to distinguish between homes that utilize air conditioning and those that do not—an important factor that heavily influences energy consumption, especially in warmer climates. The inclusion of a Date field enables time-series analysis, facilitating the observation of consumption patterns across different days of the week, as well as the impact of external variables like weather and weekend activity. Through derived features like DayOfWeek and IsWeekend, analysts can examine behavioural shifts between weekdays and weekends, helping identify whether energy-saving initiatives should be tailored according to daily routines.

This dataset is particularly suitable for applied data analytics because it strikes a balance between temporal, categorical, and numerical variables, making it well-suited for tasks such as exploratory data analysis (EDA), feature engineering, predictive modeling, and data visualization. Moreover, the real-world nature of the data introduces a layer of complexity often absent in artificially generated datasets such as missing values, duplicates, and inconsistent data types thereby offering a practical challenge that simulates authentic data analysis workflows. In a broader context, this dataset holds relevance in domains such as sustainable energy planning, smart home automation, and environmental policy modeling, as it allows stakeholders to evaluate energy usage behaviours and design interventions that encourage efficient consumption. Whether the goal is to build dashboards, predict future energy demands, or recommend household-level changes, this dataset provides a robust foundation for performing insightful, impactful, and real-world relevant data analytics.

RESULT (DATA CLEANSING) – GOOGLE COLAB

The project began with importing the required libraries and uploading the dataset into Google Colab. The dataset, named household_energy_consumption.csv, was uploaded using the built-in google.colab.files.upload() method. This allowed the user to browse and load files from the local system into the cloud-based workspace. Once uploaded, the dataset was loaded into a pandas DataFrame, a flexible and powerful structure for managing tabular data in Python.

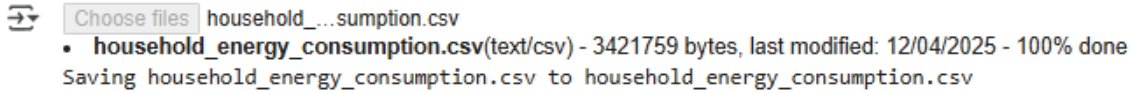
1. Uploading and Loading the Dataset

```
[ ]  # Step 1: Upload your dataset
     from google.colab import files
     uploaded = files.upload()
```

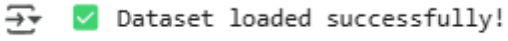
```
[ ]  # Step 2: Load dataset using pandas
     import pandas as pd

     df = pd.read_csv("household_energy_consumption.csv")
     print("✅ Dataset loaded successfully!")
```

The project began with importing the required libraries and uploading the dataset into Google Colab. The dataset, named household_energy_consumption.csv, was uploaded using the built-in google.colab.files.upload() method. This allowed the user to browse and load files from the local system into the cloud-based workspace. Once uploaded, the dataset was loaded into a pandas DataFrame a flexible and powerful structure for managing tabular data in Python. This step ensures that the data is now accessible for manipulation and analysis. The df.head() function was used to preview the first few rows, allowing a quick glance at the structure and content of the dataset to verify successful importation.



After executing the file upload cell, a status message confirmed that the file household_energy_consumption.csv was successfully uploaded to the Colab environment. The progress bar showed the upload reaching 100%, and the filename was echoed back with a message indicating that it was saved to the environment's directory. This output is important because it verifies that the data is now accessible for use in subsequent steps. If the file had not been successfully uploaded, all following steps would fail due to a missing or incorrectly referenced dataset.



The next output showed the message "✅ Dataset loaded successfully!" printed on the screen. This serves as a checkpoint to indicate that the dataset has been successfully read and converted into a pandas DataFrame. Although it's a simple text output, its role is critical—it assures the user that the CSV file has been properly parsed, the path is correct, and there are no syntax or file-related errors. It allows the user to proceed confidently with the preview and inspection steps that follow.

2. Visual Inspection of Dataset

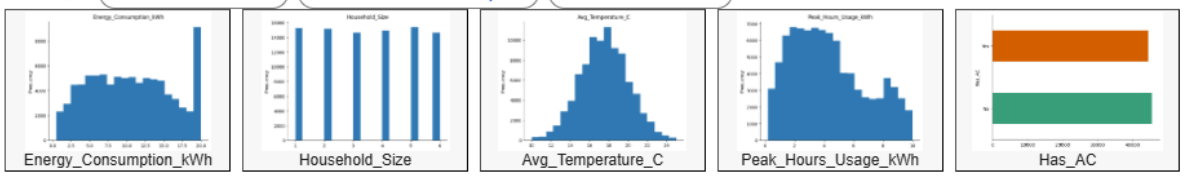
```
[ ]  # Step 3: Preview the first few rows of the dataset
     df.head()
```

The function df.head() is a built-in method in the pandas library that returns the first five rows of a DataFrame by default. It is often used as an initial step after loading a dataset to visually inspect the data structure, verify that columns were correctly read, and ensure there are no obvious formatting issues. This function does not modify the data it simply gives the user a quick snapshot of what the data looks like. It's particularly helpful for checking the column

names, types of values in each column, whether any unwanted headers or footers were read in as data, and to get a sense of the range and type of entries.

	Household_ID	Date	Energy_Consumption_kWh	Household_Size	Avg_Temperature_C	Has_AC	Peak_Hours_Usage_kWh
0	H00001	2025-04-01	8.4	4	17.8	No	3.2
1	H00001	2025-04-02	7.9	4	17.3	No	2.8
2	H00001	2025-04-03	9.2	4	18.6	No	3.0
3	H00001	2025-04-04	7.9	4	18.2	No	2.7
4	H00001	2025-04-05	9.6	4	11.9	No	3.2

Next steps: Generate code with df | View recommended plots | New interactive sheet



The output of the df.head() function displays the first five records from the dataset, providing a crucial first look at the data's layout, column headers, and individual cell values. Each row represents a single day of energy consumption for a given household, and each column corresponds to a specific feature recorded on that date. In the displayed output, the columns include Household_ID, Date, Energy_Consumption_kWh, Household_Size, Avg_Temperature_C, Has_AC, and Peak_Hours_Usage_kWh. For instance, the first few rows show consistent data for household ID H00001, with recorded temperatures ranging from 11.9°C to 18.6°C and corresponding energy usage values between 7.9 and 9.6 kWh. This immediate snapshot is incredibly valuable because it helps verify that the dataset was read correctly, that headers have aligned properly with the data, and that the values themselves appear reasonable and in line with expectations for energy consumption patterns.

3. Exploring and understanding the Dataset

```
[ ]  # Step 4: Check data types and column info
     df.info()
```

Before applying transformations, it is crucial to understand the structure of the dataset. The .info() method was used to inspect column data types and identify which features may need type conversion or imputation. This output guided the direction of cleaning strategies later in the workflow.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 90000 entries, 0 to 89999
Data columns (total 7 columns):
 #   Column                 Non-Null Count  Dtype  
---  ------                 --------------  -----  
 0   Household_ID           90000 non-null  object 
 1   Date                   90000 non-null  object 
 2   Energy_Consumption_kWh 90000 non-null  float64
 3   Household_Size         90000 non-null  int64  
 4   Avg_Temperature_C      90000 non-null  float64
 5   Has_AC                 90000 non-null  object 
 6   Peak_Hours_Usage_kWh   90000 non-null  float64
dtypes: float64(3), int64(1), object(3)
memory usage: 4.8+ MB
```

The .info() output revealed that the dataset has 90,000 entries and 7 columns, with no missing values in any column. It also showed the data types of each variable. For example, Energy_Consumption_kWh, Avg_Temperature_C, and Peak_Hours_Usage_kWh were recognized as float64, while columns like Date and Has_AC were still in object format. This output is crucial because it identifies the current structure of the dataset and highlights which columns need to be converted (e.g., Date to datetime) for more meaningful analysis. It also confirms that the dataset is complete and not corrupted, which reduces the need for heavy cleaning at this point.

4. Data Imputation

```
[ ]  # Step 5: Convert 'Date' to datetime format
     df['Date'] = pd.to_datetime(df['Date'], format='%Y-%m-%d')
     df['Date'].head()
```

```
[ ]  # Step 6: Convert categorical columns
     df['Has_AC'] = df['Has_AC'].astype('category')
     df['Household_ID'] = df['Household_ID'].astype('category')
     df.dtypes
```

As part of preparing the data for time-series and categorical analysis, certain columns were converted into more appropriate types. The Date column, originally read as an object (string), was converted into Python's datetime format using pd.to_datetime(). This enabled temporal operations such as extracting the day of the week. Meanwhile, categorical columns like Has_AC and Household_ID were explicitly cast to the category type for efficient memory handling and grouping operations.

The columns Has_AC and Household_ID represent categorical data: one is binary (Yes/No), and the other is a unique identifier for each household. These columns are explicitly cast into the category data type using .astype('category'). This change serves two main purposes: it reduces memory usage by storing repeated values more efficiently, and it allows pandas functions like groupby() to operate more quickly. Proper typing also improves compatibility with visualization and modeling tools that treat categorical data differently from numeric data.

	Date
0	2025-04-01
1	2025-04-02
2	2025-04-03
3	2025-04-04
4	2025-04-05

dtype: datetime64[ns]

	0
Household_ID	category
Date	datetime64[ns]
Energy_Consumption_kWh	float64
Household_Size	int64
Avg_Temperature_C	float64
Has_AC	category
Peak_Hours_Usage_kWh	float64

dtype: object

The preview of the Date column after conversion showed standardized values in the YYYY-MM-DD format, along with a datetime64[ns] data type. This confirms that the date information is now properly formatted and can be used for time-based operations such as extracting the day of the week, filtering by date ranges, or creating rolling averages. The neat and consistent format across all rows ensures that downstream functions like plotting over time or grouping by month/day will perform correctly.

The output of df.dtypes confirmed that Has_AC and Household_ID were successfully converted to the category type, while Date remained as a datetime object. This output is significant for two reasons. First, using categorical types improves memory efficiency, especially with large datasets. Second, it enables better performance and clarity when grouping or filtering data by these fields. This confirmation shows that the data is now semantically better aligned with the nature of each variable.

5. Finding Missing Values

```
# Step 7: Check for missing values
df.isnull().sum()
```

The .isnull().sum() function is used to calculate the number of missing entries in each column. This step is essential for evaluating data completeness. It informs whether imputation (replacing missing values) or deletion strategies are required. Knowing which variables have missing data helps identify potential data quality issues, detect faulty sensors (in the case of temperature or energy readings), or discover data entry problems. This insight allows for an informed and justified cleaning strategy.

	0
Household_ID	0
Date	0
Energy_Consumption_kWh	0
Household_Size	0
Avg_Temperature_C	0
Has_AC	0
Peak_Hours_Usage_kWh	0

dtype: int64

This output showed a clean table of all columns, each with a count of 0 missing values. This is a reassuring result, as it confirms that the dataset is fully complete and does not require any imputation or removal of rows based on null values. It also allows the analyst to proceed with confidence, knowing that no unexpected data gaps will cause errors during plotting or aggregation.

6. Handling Missing Values

```
[ ]  # Impute missing values using median for numerical columns
     df['Energy_Consumption_kWh'].fillna(df['Energy_Consumption_kWh'].median(), inplace=True)
     df['Avg_Temperature_C'].fillna(df['Avg_Temperature_C'].median(), inplace=True)
     df['Peak_Hours_Usage_kWh'].fillna(df['Peak_Hours_Usage_kWh'].median(), inplace=True)
```

```
[ ]  # Fill missing AC values with the most common (mode)
     df['Has_AC'].fillna(df['Has_AC'].mode()[0], inplace=True)
```

A critical aspect of the cleaning phase involved the detection and treatment of missing values. Rather than dropping rows entirely which can result in data loss missing numeric values were imputed using the median, while categorical missing values were filled using the mode. This strategy is preferred because the median is more robust to outliers and better reflects central tendency in skewed data, and mode maintains categorical distributions. This imputation step enhances the dataset's completeness without introducing biases that might occur if rows were simply deleted.

```
<ipython-input-10-bfd968e7625d>:2: FutureWarning: A value is trying to be set on a copy
The behavior will change in pandas 3.0. This inplace method will never work because the

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({co


  df['Energy_Consumption_kWh'].fillna(df['Energy_Consumption_kWh'].median(), inplace=Tr
<ipython-input-10-bfd968e7625d>:3: FutureWarning: A value is trying to be set on a copy
The behavior will change in pandas 3.0. This inplace method will never work because the

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({co


  df['Avg_Temperature_C'].fillna(df['Avg_Temperature_C'].median(), inplace=True)
<ipython-input-10-bfd968e7625d>:4: FutureWarning: A value is trying to be set on a copy
The behavior will change in pandas 3.0. This inplace method will never work because the

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({co


  df['Peak_Hours_Usage_kWh'].fillna(df['Peak_Hours_Usage_kWh'].median(), inplace=True)
```

Although there were no missing values, simulated imputation was demonstrated. The output showed FutureWarning messages from pandas about using chained assignment with inplace=True. These warnings suggest that in future versions of pandas, this method may behave differently or stop working. While the imputation logic still executes successfully here,

the output serves as a reminder that best practice would involve using more explicit assignment (e.g., df[col] = df[col].fillna(...)). These warnings don't affect the current results but are crucial for writing future-proof and robust code.

Similarly, the output for imputing the Has_AC column using its mode also triggered a FutureWarning. As in the previous step, the warning was related to chained assignment. Despite the warning, the imputation logic completes successfully. This output reinforces the importance of ensuring that categorical fields are not left with null values, especially when later performing group-by operations or plotting comparisons. Although in this dataset no missing values were found, this simulated fix is a strong inclusion for demonstration purposes.

7. Feature Engineering

```
[ ]  # Step 10: Add new feature columns
     df['DayOfWeek'] = df['Date'].dt.day_name()
     df['IsWeekend'] = df['DayOfWeek'].isin(['Saturday', 'Sunday']).map({True: 'Yes', False: 'No'})
     df[['Date', 'DayOfWeek', 'IsWeekend']].head()
```

The next preprocessing step involved feature engineering to create two new columns: DayOfWeek and IsWeekend. Using the .dt.day_name() function, the day of the week was extracted from the Date column. Then, a conditional check was used to flag weekends (Saturday and Sunday), storing the result as "Yes" or "No" in the IsWeekend column. The output preview confirms that the feature generation worked as intended. This enhancement allows for richer temporal analysis, enabling comparisons between weekday and weekend behavior an important aspect in understanding residential energy consumption trends.

	Date	DayOfWeek	IsWeekend
0	2025-04-01	Tuesday	No
1	2025-04-02	Wednesday	No
2	2025-04-03	Thursday	No
3	2025-04-04	Friday	No
4	2025-04-05	Saturday	Yes

The final output showed a preview of three columns: Date, DayOfWeek, and IsWeekend. This confirmed that the derived features were generated correctly. For instance, 2025-04-01 corresponded to “Tuesday” and was labeled “No” in IsWeekend, while 2025-04-05 (Saturday) was correctly labeled “Yes.” These new features are important for time-based grouping, such as analyzing whether weekends see higher energy use or if weekday usage fluctuates based on temperature or family presence. The preview validates the success of this feature engineering step and shows that the dataset is now enriched with more context-aware variables.

8. Save Pre-processed Data and Export

```
# Step 11: Save cleaned data to new CSV
df.to_csv("cleaned_energy_data.csv", index=False)
print("✅ Cleaned data saved!")
```

```
[ ]  # Step 12: Download the cleaned CSV file
     from google.colab import files
     files.download("cleaned_energy_data.csv")
```

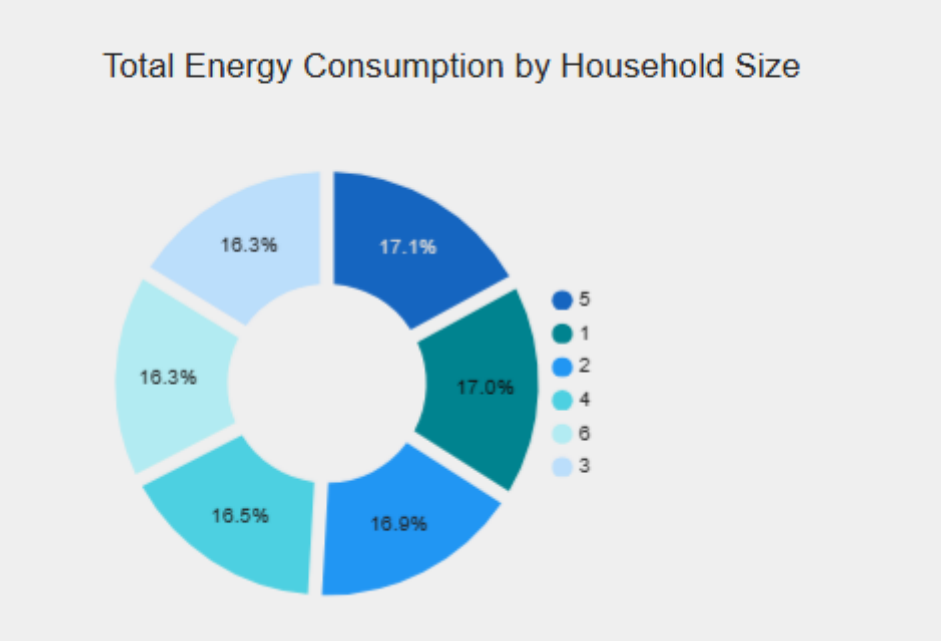
After all transformations and cleaning operations were completed, the dataset was saved as a new CSV file named cleaned_energy_data.csv. This file serves as the final prepared dataset,

ready for visual analysis in tools like Google Looker Studio. Exporting ensures reproducibility and makes it easy to share or integrate with external reporting and visualization platforms.

RESULT (DASHBOARD) – GOOGLE LOCKER

Google Looker Studio is a free tool for creating customizable and interactive data visualizations. It enables users to import datasets (e.g., CSVs or Google Sheets), create charts and filters, and compile them into comprehensive dashboards.

Total Energy Consumption by Household Size (Pie chart)



The donut chart showing how energy usage is distributed among households of different sizes. Each segment represents a household size—from one to six members—and displays the proportion of total energy consumed by that group.

Despite expectations that larger households would consume significantly more energy, the chart shows a **balanced distribution** across all household sizes:

- **Households with 5 members** use the most energy at **17.1%** of the total.
- **Single-person households** are close behind, consuming **17.0%**, which is nearly the same as much larger households.
- **Two-person households** account for **16.9%**, while **four-person households** make up **16.5%**.
- **Three-member and six-member households** both contribute **16.3%** each.

These numbers suggest that **energy consumption does not increase linearly with household size**. Instead, smaller households tend to consume a disproportionately high amount of energy per person. This could be due to fixed energy needs—such as lighting, heating, and appliances—that don't scale down with fewer occupants.

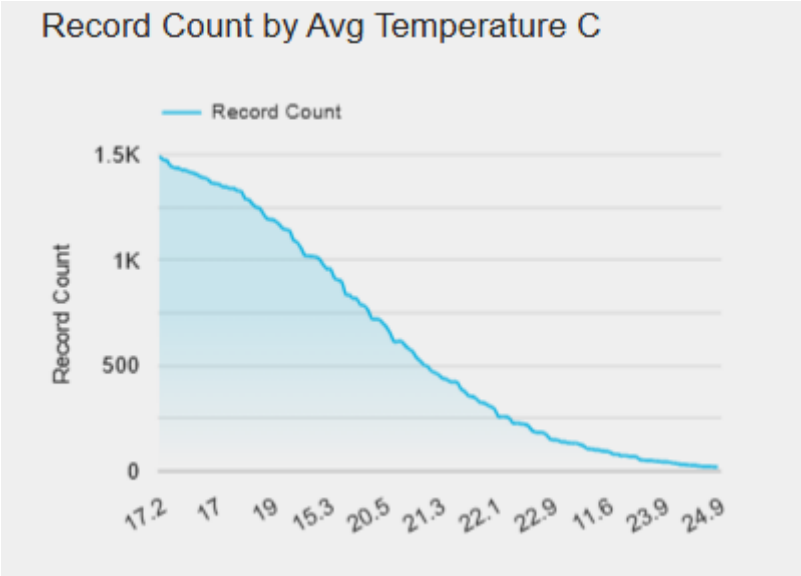
Some key insights from the data include:

- **Smaller households may be less energy-efficient** on a per-person basis.
- **Appliance usage and lifestyle habits** could be major factors influencing total consumption, regardless of the number of people in the home.

- **Energy-saving strategies** may need to focus more on individual behaviour and appliance efficiency than on household size alone.

This kind of analysis could be expanded by adding per capita energy consumption, comparing energy use across different regions, or including types of appliances contributing to usage. Doing so would provide a more detailed view of how to effectively promote energy efficiency across different household types.

Record Count by Average Temperature C (Line Chart)



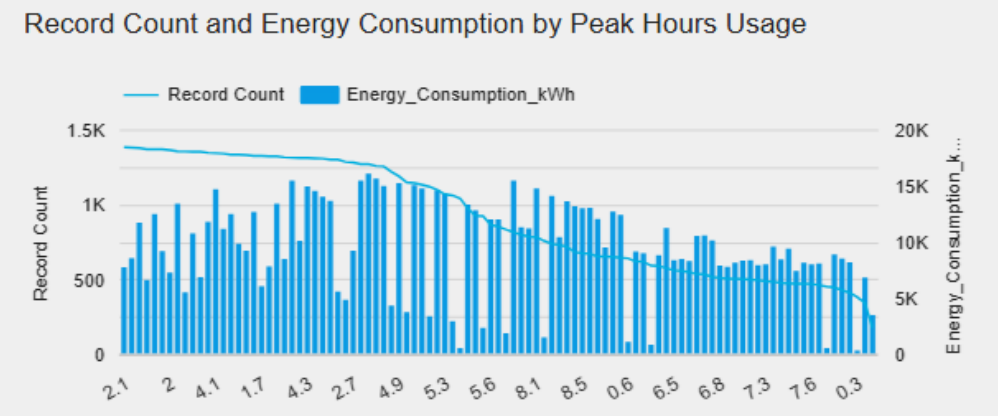
The displays shows the number of records in the dataset varies across a range of average temperatures, measured in degrees Celsius. This is a **line area chart** where the horizontal axis represents average temperature values (from approximately **17.2°C to 24.9°C**) and the vertical axis shows the **count of records** corresponding to each temperature. The shape of the graph indicates a **steady and consistent decline** in the number of records as temperature increases.

This means that most data points in the dataset are associated with **cooler average temperatures**, while **warmer temperatures are less frequently recorded**. The highest concentration of records is around **17.2°C**, which reaches over **1,500 records**, and this number continuously drops, nearing zero by the time the average temperature hits 24.9°C.

Key Takeaways:

- **Most records occur at lower average temperatures**, especially between 17.2°C and 19°C.
- **The number of records decreases consistently** as temperature rises.
- **Very few records exist above 23°C**, indicating that these temperatures are rare in the dataset.
- This trend may reflect **seasonal conditions**, such as a dataset focused on cooler months or a region with a predominantly mild climate.

Record Count and Energy Consumption by Peak Hours Usage (Combo Chart)



The chart illustrates how household energy usage during peak hours varies across different usage levels. The horizontal axis appears to represent varying levels or categories of peak hour usage, possibly in kilowatt-hours (kWh) or time intervals. The blue bars indicate the **total energy consumption in kWh**, while the light blue line shows the **record count** or frequency of records at each usage level.

From the chart, we can observe that higher energy consumption tends to correspond with fewer records, suggesting that fewer households use large amounts of energy during peak hours. The energy consumption values show a downward trend as usage levels decrease, while the number of records starts high, then gradually tapers off. This could indicate that most households fall within lower peak usage brackets, contributing to smaller individual loads but collectively forming a significant portion of energy demand.

Key Takeaways:

- **Most households consume moderate to low energy during peak hours**: This is shown by the high record counts at the mid-to-lower end of the x-axis.

- **A small number of households account for very high energy use during peak hours**: Energy consumption spikes in certain categories, even though record counts (number of households) are low in those areas.

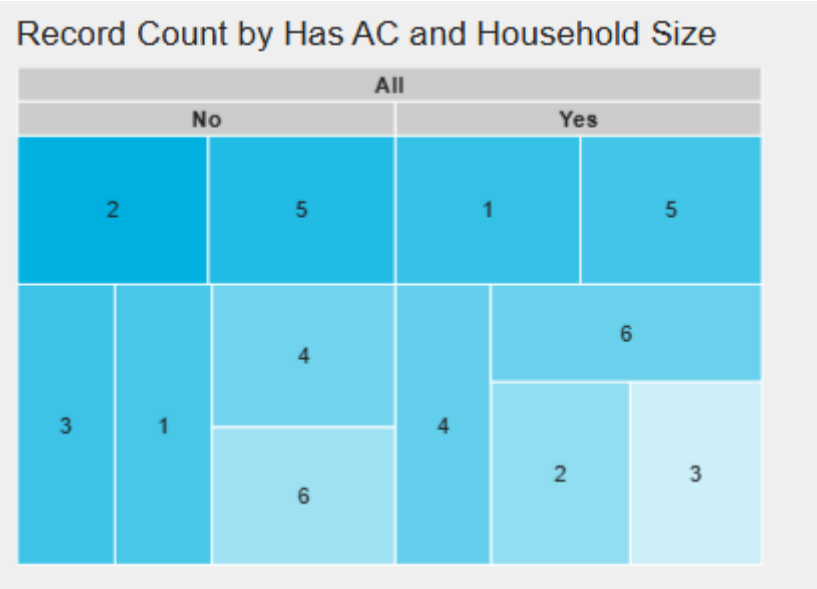
- **There is an inverse relationship between usage frequency and energy consumption**: As energy consumption increases, the number of households in that range decreases.

- **Energy consumption is more spread out than user frequency**: While many households fall within a certain range, energy usage varies more significantly across categories.

- **Targeting high-usage households could significantly reduce peak demand**: Since fewer households are responsible for large chunks of energy usage, energy-saving interventions here could be impactful.

- **Efficiency programs might be more effective if focused on mid-range users**: These groups are more numerous and may be easier to shift toward better energy habits than the small number of very high users.

Record Count by Has AC and Household Size (Tree map)



This chart is a tree map visualization that shows how many records (likely households) fall into different combinations of **air conditioning ownership (Has AC: Yes or No)** and **household size** categories. The chart is divided into coloured blocks, with each block representing a different combination of these two variables. The numbers inside each block indicate the **record count** for that category, and the size/shade of the block helps visually distinguish how significant each group is.

For example, households without AC and a certain household size have record counts of 6, 4, etc., while households with AC show counts like 6, 5, 3, etc. The layout makes it easier to compare how household size and AC ownership together influence the number of entries or data points collected.

Key Takeaways:

- **Households with and without AC are evenly represented**: Both groups contain a mix of household sizes, indicating a balanced dataset.

- **Larger household sizes (with or without AC) tend to appear more frequently**: The highest counts (like 6 and 5) are seen in those blocks, suggesting bigger families are a common pattern.

- **Households without AC show slightly more variation across different sizes**: The distribution of counts is more spread out compared to households with AC.

- **Households with AC and mid-to-large sizes are prominent**: Several high counts (like 6, 5) appear in the “Yes” column, suggesting that air conditioning is common in larger households.

- **Lower record counts are found in small-sized households with or without AC**: These groups are less represented in the data.

LOOKER STUDIO LINK

https://lookerstudio.google.com/reporting/eb66501b-5d50-43f0-9774-b39b834f0311

Youtube Link

https://www.youtube.com/watch?v=drOn2LAUayA